



8.1. Singly Linked Lists

Previous Section:

 [8. Linked Lists](#)

Contents:

[1. Traversal](#)

[2. Length of a Linked List](#)

[3. Searching](#)

[4. Insertion and Deletion](#)

[4.1. Insertion at the Beginning](#)

[4.2. Insertion at the End](#)

[4.3. Insertion at a Certain Position](#)

[4.4. Deletion at the Beginning](#)

[4.5. Deletion at the End](#)

[4.6. Deletion at a Certain Position](#)

[5. Insertion in Ascending Order](#)

[Possible Cases](#)

[Summary](#)

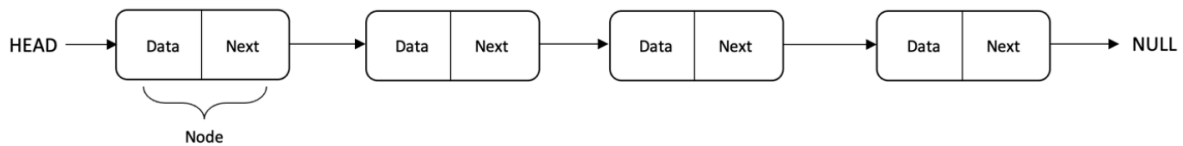
[References](#)

Now that we understand what linked lists are and why they are useful, let us begin with the most fundamental type: the **Singly Linked List**.

A Singly Linked List is a collection of nodes where each node stores two pieces of information: **A data value** and **a reference to the next node**.

Unlike arrays, nodes are not stored next to each other in memory. Instead, each node simply knows where the next node is located. This creates a chain-like structure that can grow or shrink dynamically as needed.

For example, a singly linked list containing four values might look like this:



The first node is called the **Head** of the list. The final node points to **NULL**, indicating the end of the chain.

Before we implement linked list operations, we first need two classes:

- A **Node** class to represent an individual node.
- A **SinglyLinkedList** class to manage the entire linked list.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None
```

The `Node` class stores the value and a reference to the next node. The `SinglyLinkedList` class stores the head of the list, which serves as the starting point for all operations.

1. Traversal

Traversal is the process of visiting every node in the linked list.

Since a singly linked list only stores references to the next node, we must start from the head and move forward one node at a time until we reach NULL.

Implementation:

```
def traverse(self):
    # Empty list
    if self.head is None:
        print("List is empty")
    else:
        temp = self.head
        while temp:
            print(temp.data, end=" => ")
            temp = temp.next
        print("None")
```

2. Length of a Linked List

Unlike arrays, linked lists do not automatically know how many elements they contain.

To determine the size of a linked list, we must traverse the entire list and count the nodes one by one.

Implementation:

```
def get_size(self):
    # Return length of the linked list
    count = 0
    temp = self.head

    while temp:
        count += 1
        temp = temp.next

    return count
```

3. Searching

Searching means locating a particular value inside the linked list.

Since nodes cannot be accessed directly by index, we must traverse the list until the desired value is found.

Implementation:

```
def search(self, data):
    # Search the value and return its position
    temp = self.head
    position = 1

    while temp:
        if temp.data == data:
            return position

        temp = temp.next
        position += 1

    return -1
```

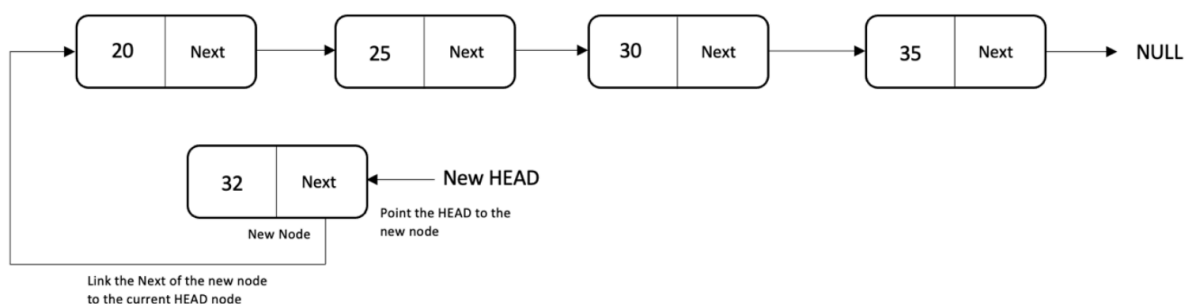
4. Insertion and Deletion

4.1. Insertion at the Beginning

Insertion at the beginning is one of the fastest linked list operations.

The process is simple:

1. Create a new node.
2. Point the new node to the current head.
3. Update the head so that it points to the new node.



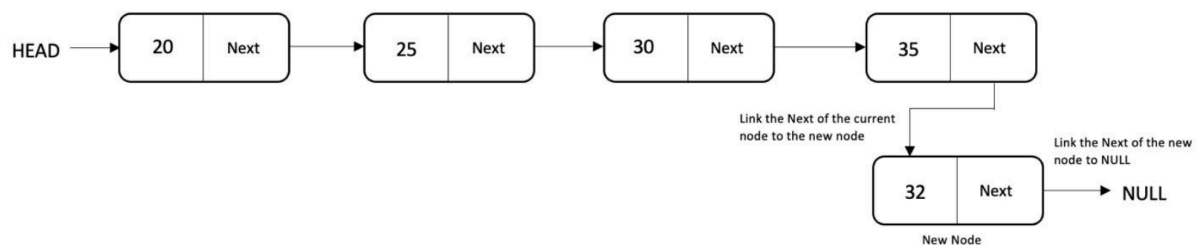
Time Complexity: $O(1)$

Implementation:

```
def insert_begin(self, data):  
    new_node = Node(data)  
    new_node.next = self.head  
    self.head = new_node
```

4.2. Insertion at the End

To insert a node at the end of the list, we must first locate the current last node. The new node is then attached after it.



Time Complexity: $O(n)$

Implementation:

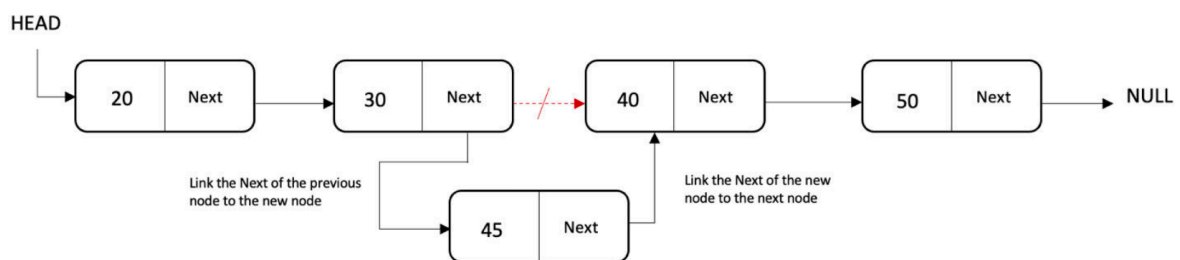
```
def insert_end(self, data):  
    new_node = Node(data)  
  
    if self.head is None:  
        self.head = new_node  
    else:  
        temp = self.head  
  
        while temp.next:  
            temp = temp.next  
  
        temp.next = new_node
```

4.3. Insertion at a Certain Position

Sometimes we want to insert a node between two existing nodes.

The process involves:

1. Creating a new node.
2. Locating the node before the insertion point.
3. Connecting the new node to the next node.
4. Connecting the previous node to the new node.



Time Complexity: $O(n)$

Implementation:

```
def insert_certain_position(self, data, position):
    if position < 1:
        print("Position is invalid")
        return

    new_node = Node(data)

    if position == 1:
        new_node.next = self.head
        self.head = new_node
    else:
        temp = self.head

        for i in range(position - 2):
            if temp is None:
                print("Position is invalid")
                return
```

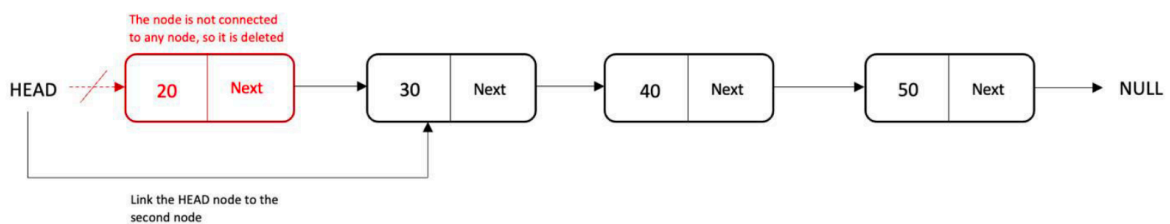
```
temp = temp.next
```

```
new_node.next = temp.next
```

```
temp.next = new_node
```

4.4. Deletion at the Beginning

Deleting the first node is very straightforward. We simply move the head reference to the second node.



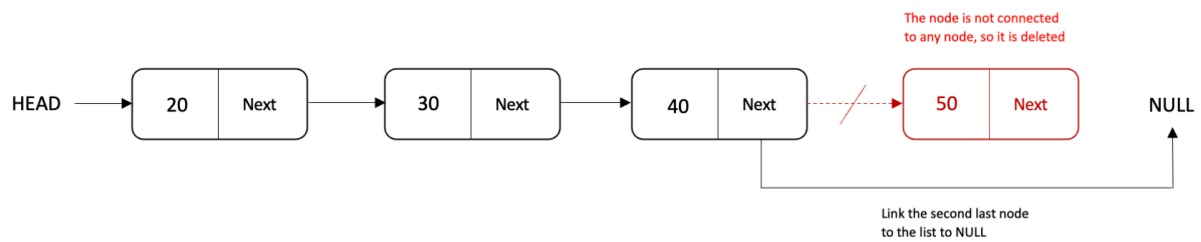
Time Complexity: $O(1)$

Implementation:

```
def delete_begin(self):  
    if self.head is None:  
        print("List is empty")  
        return  
  
    self.head = self.head.next
```

4.5. Deletion at the End

Deleting the last node requires locating the second-to-last node. Its next reference is then updated to NULL.



Special cases include **empty list** or **list containing only one node**.

Time Complexity: $O(n)$

Implementation:

```
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next is None:
        self.head = None
        return

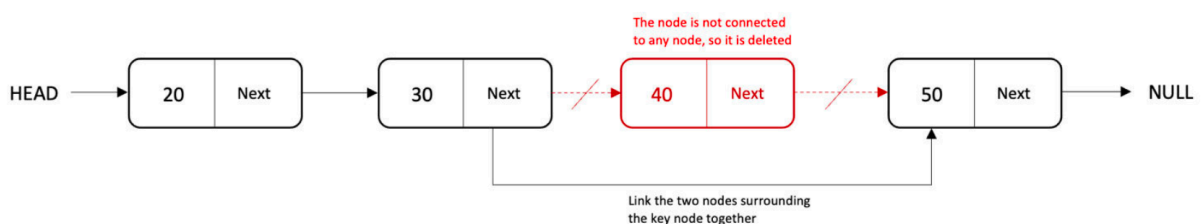
    temp = self.head

    while temp.next and temp.next.next:
        temp = temp.next

    temp.next = None
```

4.6. Deletion at a Certain Position

To remove a node from a specific position, we reconnect the surrounding nodes so that the target node is skipped.



Implementation:

```
def delete_certain_position(self, position):
    if position < 1:
        print("Position is invalid")
        return

    if self.head is None:
        print("List is empty")
        return

    if position == 1:
        self.head = self.head.next
        return

    temp = self.head

    for i in range(position - 2):
        if temp is None or temp.next is None:
            print("Position is invalid")
            return
        temp = temp.next

    if temp.next is None:
        print("Position is invalid")
        return

    temp.next = temp.next.next
```

Time Complexity: $O(n)$

Below is the full functional Singly Linked List implementation:

[singly linked lists.py](#)

At this point, we can perform all fundamental operations on a Singly Linked List: traversal, searching, insertion, and deletion. While the structure is simple and memory-efficient, it also comes with a limitation: nodes can only be traversed in one direction.

In the next section, we will explore **Doubly Linked Lists**, which extend this design by allowing movement both forward and backward through the list, making certain operations even more convenient and efficient.

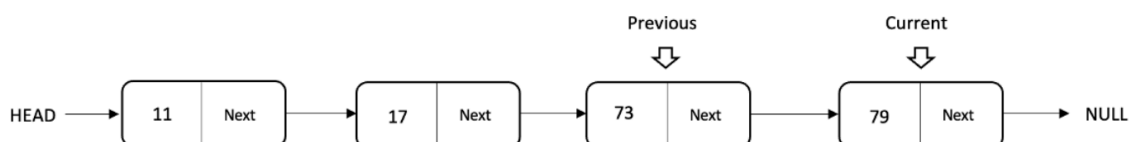
5. Insertion in Ascending Order

So far, the insertion operations we have studied require us to specify where the new node should be inserted. However, in many applications, we want the linked list to remain sorted automatically.

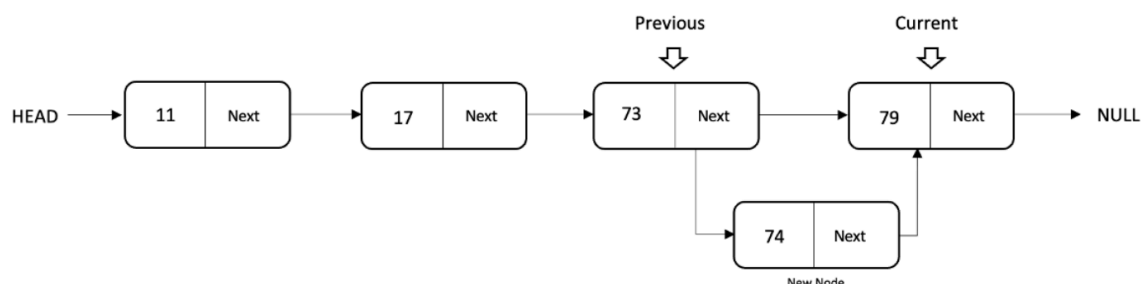
Instead of manually finding the correct position every time, we can improve our insertion algorithm so that each new node is inserted into its proper location while preserving the ascending order of the list.

For instance, if we insert **30**, the algorithm should automatically place it between **25** and **40**.

Before insertion in mid of linked list (sorted)



After insertion in mid of linked list (sorted)



The basic idea is simple:

1. Create a new node.

2. Traverse the linked list until the correct insertion position is found.
3. Keep track of both the current node and the previous node.
4. Insert the new node between them.
5. If the new value is smaller than the current head, insert it at the beginning.
6. If the new value is larger than all existing nodes, insert it at the end.

This approach works for both **Singly Linked Lists** and **Doubly Linked Lists**.

Possible Cases

When inserting into a sorted linked list, several situations may occur:

- **Case 1 - Insert at the Beginning:** The new node becomes the new head.
- **Case 2 - Insert in the Middle:** The new node is inserted between two existing nodes.
- **Case 3 - Insert at the End.** The new node becomes the new tail of the list.
- **Case 4 - Empty List.** The new node becomes the head of the list.

To locate the correct insertion position, we may need to traverse the entire linked list. Therefore, the **time complexity is $O(n)$** , where n is the number of nodes in the linked list.

The following example automatically inserts random values while maintaining ascending order.

```

import random

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class LinkedList:
    def __init__(self):
        self.head = None

    def traverse(self):
        if self.head is None:
            print("List is empty")
        else:
            temp = self.head

            while temp:
                print(temp.data, end=" => ")
                temp = temp.next

            print("None")

    def insert(self, data):
        temp = Node(data)

        # Empty list
        if self.head is None:
            self.head = temp
            return

        # Insert at beginning
        if temp.data <= self.head.data:
            temp.next = self.head
            self.head = temp

```

```

        return

    previous = self.head
    current = self.head

    while (current is not None) and (current.data < temp
p.data):
        previous = current
        current = current.next

    temp.next = current
    previous.next = temp

def main():
    linked_list = LinkedList()
    num_nodes = int(input("Enter number of nodes: "))

    for n in range(num_nodes):
        node = random.randint(0, 100)
        linked_list.insert(node)

    linked_list.traverse()

main()

```

Sample Output - Suppose the random values generated are:
75, 20, 90, 35, 10, 60

Although the values are generated in random order, the linked list automatically stores them in ascending order:

10 ⇒ 20 ⇒ 35 ⇒ 60 ⇒ 75 ⇒ 90 ⇒ None

Summary

In this section, we explored the **Singly Linked List**, the simplest and most fundamental form of linked list. We learned how nodes are connected using a

single reference to the next node and how this structure allows dynamic memory allocation without requiring contiguous storage.

We implemented the most important linked list operations, including traversal, searching, calculating the length of the list, insertion, and deletion. We also extended the insertion operation by maintaining the linked list in ascending order automatically, allowing new nodes to be placed in their correct sorted positions.

Throughout these operations, we observed one major characteristic of a singly linked list: every node only knows the location of the next node. As a result, traversal is possible only in the forward direction. While this keeps the structure simple and memory-efficient, certain operations become less convenient because moving backward through the list is impossible.

To address this limitation, computer scientists developed another variation of linked lists that stores additional information in each node. In the next section, we will study the **Doubly Linked List**, where each node maintains references to both the next node and the previous node, allowing efficient traversal in both directions and simplifying many insertion and deletion operations.

References

https://drive.google.com/file/d/1xW6GD_okYUEMMlotBxclA5RdAe2Rx8Gu/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/singly-linked-list-tutorial/>

<https://www.datacamp.com/tutorial/python-linked-lists>

<https://www.programiz.com/dsa/linked-list>

Next Section:

 [8.2. Doubly Linked Lists](#)